Route Parameters and Query Strings in Routing

Back to ASP.NET Core Web APIs Tutorials

Route Parameters and Query Strings in ASP.NET Core Web API

Route parameters and Query Strings in ASP.NET Core Web API Tutorial. Route parameters are used to pass data in ASP.NET Core Web API Application. When developing **RESTful** services using ASP.NET Core Web API, we need to pass data to the server using the following ways.

- Writing data directly in the URL
- Passing a query string in URL
- Sending the parameters in header

ASP.NET Core provides two ways to access such data.

- **Route Data**
- **Query Strings**

Both Route Data and Query Strings are **key-value pairs stored in the Request URL**. The URL will look like **QueryStringExample**

Route Data is a part of the URL and **Query Strings** is a part of the URL. Route Data is **mandatory** and **Query Strings** is **optional**. Route Data is used when the URL is **mandatory** for that resource to function. It is also preferred to **use Route Data when the URL is more readable than Query Strings**.

Route Data is used to pass information as part of the URL path. It identifies a specific resource or a collection of resources, such as a Employee or Order. Route Data is **mandatory**.

Query parameters are embedded directly in the URL and are **mandatory** for that resource to function. It is also preferred to **use Query Strings when the URL is more readable than Route Data**. Query Strings are **optional**.

Route Data is a part of the URL path. It identifies a specific resource or a collection of resources, such as a Employee or Order. Route Data is **mandatory**.

Query parameters are embedded directly in the URL and are **mandatory** for that resource to function. It is also preferred to **use Query Strings when the URL is more readable than Route Data**. Query Strings are **optional**.

Route Data is mandatory and **Query Strings** is optional.

Route Data is used when the URL is **mandatory** for that resource to function. It is also preferred to **use Route Data when the URL is more readable than Query Strings**.

Query Strings is used when the URL is **optional** for that resource to function. It is also preferred to **use Query Strings when the URL is more readable than Route Data**.

Example to Understand Route Data in ASP.NET Core Web API

Create a new Web **Application (Model View Controller)** project with the name **RouteDataDemo**.

Define the Employee Model

Create a folder by name **Models** and add a **Employee.cs** Model file and then copy and paste the following code.

```
namespace RouteDataDemo.Models
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public int Salary { get; set; }
        public int Age { get; set; }
        public bool IsPermanent { get; set; }
        public int DeptID { get; set; }
        public DateTime? DateOfJoining { get; set; }
        public static List<Employee> GetEmployeeDetails()
        {
            return new List<Employee>
            {
                new Employee { ID = 1, Name = "Rajesh", Salary = 5000, Age = 30, IsPermanent = true, DeptID = 1, DateOfJoining = DateTime.Parse("01/01/2010") },
                new Employee { ID = 2, Name = "Rakesh", Salary = 6000, Age = 32, IsPermanent = false, DeptID = 2, DateOfJoining = DateTime.Parse("02/02/2011") },
                new Employee { ID = 3, Name = "Suresh", Salary = 7000, Age = 34, IsPermanent = true, DeptID = 3, DateOfJoining = DateTime.Parse("03/03/2012") },
                new Employee { ID = 4, Name = "Ganesh", Salary = 8000, Age = 36, IsPermanent = false, DeptID = 4, DateOfJoining = DateTime.Parse("04/04/2013") },
                new Employee { ID = 5, Name = "Vijay", Salary = 9000, Age = 38, IsPermanent = true, DeptID = 5, DateOfJoining = DateTime.Parse("05/05/2014") }
            };
        }
    }
}
```

Initialize in Memory Data

Create a folder by name **RepositoryData** and add the **Models** folder and then copy and paste the following code.

```
namespace RouteDataDemo.RepositoryData
{
    public class EmployeeRepository
    {
        private static IEnumerable<Employee> _emp = Employee.GetEmployeeDetails();
        public IEnumerable<Employee> GetAllEmployees()
        {
            return _emp;
        }
        public Employee GetEmployeeByID(int id)
        {
            return _emp.FirstOrDefault(e => e.ID == id);
        }
        public IEnumerable<Employee> GetEmployeesByDeptID(int deptid)
        {
            return _emp.Where(e => e.DeptID == deptid);
        }
        public IEnumerable<Employee> GetEmployeesBetweenAge(int a1, int a2)
        {
            return _emp.Where(e => e.Age > a1 && e.Age < a2);
        }
        public IEnumerable<Employee> GetEmployeesBetweenDateOfJoining(DateTime? d1, DateTime? d2)
        {
            return _emp.Where(e => e.DateOfJoining > d1 && e.DateOfJoining < d2);
        }
        public IEnumerable<Employee> GetPermanentEmployees(bool ispermanent)
        {
            return _emp.Where(e => e.IsPermanent == ispermanent);
        }
        public IEnumerable<Employee> GetEmployeesBetweenSalary(int s1, int s2)
        {
            return _emp.Where(e => e.Salary > s1 && e.Salary < s2);
        }
        public int AddEmployee(Employee emp)
        {
            _emp.Add(emp);
            return emp.ID;
        }
        public int UpdateEmployee(Employee emp)
        {
            var e = _emp.FirstOrDefault(x => x.ID == emp.ID);
            if (e != null)
            {
                e.Name = emp.Name;
                e.Salary = emp.Salary;
                e.Age = emp.Age;
                e.IsPermanent = emp.IsPermanent;
                e.DeptID = emp.DeptID;
                e.DateOfJoining = emp.DateOfJoining;
                return e.ID;
            }
            else
            {
                return -1;
            }
        }
        public int DeleteEmployee(int id)
        {
            var e = _emp.FirstOrDefault(x => x.ID == id);
            if (e != null)
            {
                _emp.Remove(e);
                return e.ID;
            }
            else
            {
                return -1;
            }
        }
    }
}
```

Fetching Employees by ID using Route Data

Add a new Controller by name **EmployeeRouteDataController** to the **Controllers** folder and then copy and paste the following code. In the action method, we are using the parameter **id** to get the employee details. In the action method, we are using the **EmployeeRepository** class to get the employee details. We are using the **ViewBag** to pass the employee data to the view.

```
namespace RouteDataDemo.Controllers
{
    public class EmployeeRouteDataController : Controller
    {
        EmployeeRepository _repo = new EmployeeRepository();
        public IEnumerable<Employee> _emp = Employee.GetEmployeeDetails();
        // GET: /EmployeeRouteData/
        public ActionResult Index(int id = 0)
        {
            if (id > 0)
            {
                var e = _repo.GetEmployeeByID(id);
                if (e != null)
                {
                    ViewBag.Message = e.ID + " " + e.Name + " " + e.Salary + " " + e.Age + " " + e.IsPermanent + " " + e.DeptID + " " + e.DateOfJoining;
                }
                else
                {
                    ViewBag.Message = "Employee not found";
                }
            }
            else
            {
                ViewBag.Message = "Please pass a valid Employee id";
            }
            return View();
        }
    }
}
```

Employee Controller

We always modify the Employee Controller code as shown below.

```
namespace RouteDataDemo.Controllers
{
    public class EmployeeController : Controller
    {
        EmployeeRepository _repo = new EmployeeRepository();
        public IEnumerable<Employee> _emp = Employee.GetEmployeeDetails();
        // GET: /Employee/
        public ActionResult Index(int id = 0)
        {
            if (id > 0)
            {
                var e = _repo.GetEmployeeByID(id);
                if (e != null)
                {
                    ViewBag.Message = e.ID + " " + e.Name + " " + e.Salary + " " + e.Age + " " + e.IsPermanent + " " + e.DeptID + " " + e.DateOfJoining;
                }
                else
                {
                    ViewBag.Message = "Employee not found";
                }
            }
            else
            {
                ViewBag.Message = "Please pass a valid Employee id";
            }
            return View();
        }
        public ActionResult GetEmployeeByID(int id)
        {
            var e = _repo.GetEmployeeByID(id);
            if (e != null)
            {
                ViewBag.Message = e.ID + " " + e.Name + " " + e.Salary + " " + e.Age + " " + e.IsPermanent + " " + e.DeptID + " " + e.DateOfJoining;
                return View("Index");
            }
            else
            {
                ViewBag.Message = "Employee not found";
                return View("Index");
            }
        }
        public ActionResult GetEmployeesByDeptID(int deptid)
        {
            var e = _repo.GetEmployeesByDeptID(deptid);
            if (e != null)
            {
                ViewBag.Message = e.Count();
                return View("Index");
            }
            else
            {
                ViewBag.Message = "Employee not found";
                return View("Index");
            }
        }
        public ActionResult GetEmployeesBetweenAge(int a1, int a2)
        {
            var e = _repo.GetEmployeesBetweenAge(a1, a2);
            if (e != null)
            {
                ViewBag.Message = e.Count();
                return View("Index");
            }
            else
            {
                ViewBag.Message = "Employee not found";
                return View("Index");
            }
        }
        public ActionResult GetEmployeesBetweenDateOfJoining(DateTime? d1, DateTime? d2)
        {
            var e = _repo.GetEmployeesBetweenDateOfJoining(d1, d2);
            if (e != null)
            {
                ViewBag.Message = e.Count();
                return View("Index");
            }
            else
            {
                ViewBag.Message = "Employee not found";
                return View("Index");
            }
        }
        public ActionResult GetPermanentEmployees(bool ispermanent)
        {
            var e = _repo.GetPermanentEmployees(ispermanent);
            if (e != null)
            {
                ViewBag.Message = e.Count();
                return View("Index");
            }
            else
            {
                ViewBag.Message = "Employee not found";
                return View("Index");
            }
        }
        public ActionResult GetEmployeesBetweenSalary(int s1, int s2)
        {
            var e = _repo.GetEmployeesBetweenSalary(s1, s2);
            if (e != null)
            {
                ViewBag.Message = e.Count();
                return View("Index");
            }
            else
            {
                ViewBag.Message = "Employee not found";
                return View("Index");
            }
        }
        public int AddEmployee(Employee emp)
        {
            _emp.Add(emp);
            return emp.ID;
        }
        public int UpdateEmployee(Employee emp)
        {
            var e = _emp.FirstOrDefault(x => x.ID == emp.ID);
            if (e != null)
            {
                e.Name = emp.Name;
                e.Salary = emp.Salary;
                e.Age = emp.Age;
                e.IsPermanent = emp.IsPermanent;
                e.DeptID = emp.DeptID;
                e.DateOfJoining = emp.DateOfJoining;
                return e.ID;
            }
            else
            {
                return -1;
            }
        }
        public int DeleteEmployee(int id)
        {
            var e = _emp.FirstOrDefault(x => x.ID == id);
            if (e != null)
            {
                _emp.Remove(e);
                return e.ID;
            }
            else
            {
                return -1;
            }
        }
    }
}
```

Code Explanation

- **Route Parameter**: Route parameters are used to pass data from URL path. Example: /Employee/GetEmployeeByID/5 → id = 5
- **Route Constraint**: Ensures the route matches only when the parameter is a valid integer (int).
- **Action Method**: GetEmployeeByID(int id) receives the id from the URL and uses it to fetch the employee.
- **Data Passing**: The employee data is passed to the view using ViewBag.Message.
- **Conditional Logic**: The code checks if the employee exists before displaying the data.

Testing the Endpoint

Run the application and navigate to the URL path. Provide a valid Employee id. If the employee exists, the details will be displayed. If the employee does not exist, a message indicating that the employee is not found will be displayed.

```
http://localhost:7154/Employee/GetEmployeeByID/1
```

Fetching Employees by DeptID using Route Data

Add a new action method by name **GetEmployeesByDeptID** to the **EmployeeController** and then copy and paste the following code. In the action method, we are using the parameter **deptid** to get the employee details. In the action method, we are using the **EmployeeRepository** class to get the employee details. We are using the **ViewBag** to pass the employee data to the view.

```
public ActionResult GetEmployeesByDeptID(int deptid)
{
    var e = _repo.GetEmployeesByDeptID(deptid);
    if (e != null)
    {
        ViewBag.Message = e.Count();
        return View("Index");
    }
    else
    {
        ViewBag.Message = "Employee not found";
        return View("Index");
    }
}
```

Run the application and navigate to the URL path. Provide a valid Department id. If the employee exists, the details will be displayed. If the employee does not exist, a message indicating that the employee is not found will be displayed.

```
http://localhost:7154/Employee/GetEmployeesByDeptID/1
```

Fetching Employees between Age using Route Data

Add a new action method by name **GetEmployeesBetweenAge** to the **EmployeeController** and then copy and paste the following code. In the action method, we are using the parameters **a1** and **a2** to get the employee details. In the action method, we are using the **EmployeeRepository** class to get the employee details. We are using the **ViewBag** to pass the employee data to the view.

```
public ActionResult GetEmployeesBetweenAge(int a1, int a2)
{
    var e = _repo.GetEmployeesBetweenAge(a1, a2);
    if (e != null)
    {
        ViewBag.Message = e.Count();
        return View("Index");
    }
    else
    {
        ViewBag.Message = "Employee not found";
        return View("Index");
    }
}
```

Run the application and navigate to the URL path. Provide valid Age values. If the employee exists, the details will be displayed. If the employee does not exist, a message indicating that the employee is not found will be displayed.

```
http://localhost:7154/Employee/GetEmployeesBetweenAge/30/40
```

Fetching Employees between DateOfJoining using Route Data

Add a new action method by name **GetEmployeesBetweenDateOfJoining** to the **EmployeeController** and then copy and paste the following code. In the action method, we are using the parameters **d1** and **d2** to get the employee details. In the action method, we are using the **EmployeeRepository** class to get the employee details. We are using the **ViewBag** to pass the employee data to the view.

```
public ActionResult GetEmployeesBetweenDateOfJoining(DateTime? d1, DateTime? d2)
{
    var e = _repo.GetEmployeesBetweenDateOfJoining(d1, d2);
    if (e != null)
    {
        ViewBag.Message = e.Count();
        return View("Index");
    }
    else
    {
        ViewBag.Message = "Employee not found";
        return View("Index");
    }
}
```

Run the application and navigate to the URL path. Provide valid DateOfJoining values. If the employee exists, the details will be displayed. If the employee does not exist, a message indicating that the employee is not found will be displayed.

```
http://localhost:7154/Employee/GetEmployeesBetweenDateOfJoining/01-01-2010/01-01-2015
```


Fetching Permanent Employees using Route Data

Add a new action method by name **GetPermanentEmployees** to the **EmployeeController** and then copy and paste the following code. In the action method, we are using the parameter **ispermanent** to get the employee details. In the action method, we are using the **EmployeeRepository** class to get the employee details. We are using the **ViewBag** to pass the employee data to the view.

```
public ActionResult GetPermanentEmployees(bool ispermanent)
{
    var e = _repo.GetPermanentEmployees(ispermanent);
    if (e != null)
    {
        ViewBag.Message = e.Count();
        return View("Index");
    }
    else
    {
        ViewBag.Message = "Employee not found";
        return View("Index");
    }
}
```

Run the application and navigate to the URL path. Provide valid ispermanent values. If the employee exists, the details will be displayed. If the employee does not exist, a message indicating that the employee is not found will be displayed.

```
http://localhost:7154/Employee/GetPermanentEmployees/true
```

Fetching Employees between Salary using Route Data

Add a new action method by name **GetEmployeesBetweenSalary** to the **EmployeeController** and then copy and paste the following code. In the action method, we are using the parameters **s1** and **s2** to get the employee details. In the action method, we are using the **EmployeeRepository** class to get the employee details. We are using the **ViewBag** to pass the employee data to the view.

```
public ActionResult GetEmployeesBetweenSalary(int s1, int s2)
{
    var e = _repo.GetEmployeesBetweenSalary(s1, s2);
    if (e != null)
    {
        ViewBag.Message = e.Count();
        return View("Index");
    }
    else
    {
        ViewBag.Message = "Employee not found";
        return View("Index");
    }
}
```

Run the application and navigate to the URL path. Provide valid Salary values. If the employee exists, the details will be displayed. If the employee does not exist, a message indicating that the employee is not found will be displayed.

```
http://localhost:7154/Employee/GetEmployeesBetweenSalary/5000/8000
```

Modifying Employee Controller

Please modify the Employee Controller code as shown below.

```
namespace RouteDataDemo.Controllers
{
    public class EmployeeController : Controller
    {
        EmployeeRepository _repo = new EmployeeRepository();
        public IEnumerable<Employee> _emp = Employee.GetEmployeeDetails();
        // GET: /Employee/
        public ActionResult Index(int id = 0)
        {
            if (id > 0)
            {
                var e = _repo.GetEmployeeByID(id);
                if (e != null)
                {
                    ViewBag.Message = e.ID + " " + e.Name + " " + e.Salary + " " + e.Age + " " + e.IsPermanent + " " + e.DeptID + " " + e.DateOfJoining;
                }
                else
                {
                    ViewBag.Message = "Employee not found";
                }
            }
            else
            {
                ViewBag.Message = "Please pass a valid Employee id";
            }
            return View();
        }
        public ActionResult GetEmployeesByDeptID(int deptid)
        {
            var e = _repo.GetEmployeesByDeptID(deptid);
            if (e != null)
            {
                ViewBag.Message = e.Count();
                return View("Index");
            }
            else
            {
                ViewBag.Message = "Employee not found";
                return View("Index");
            }
        }
        public ActionResult GetEmployeesBetweenAge(int a1, int a2)
        {
            var e = _repo.GetEmployeesBetweenAge(a1, a2);
            if (e != null)
            {
                ViewBag.Message = e.Count();
                return View("Index");
            }
            else
            {
                ViewBag.Message = "Employee not found";
                return View("Index");
            }
        }
        public ActionResult GetEmployeesBetweenDateOfJoining(DateTime? d1, DateTime? d2)
        {
            var e = _repo.GetEmployeesBetweenDateOfJoining(d1, d2);
            if (e != null)
            {
                ViewBag.Message = e.Count();
                return View("Index");
            }
            else
            {
                ViewBag.Message = "Employee not found";
                return View("Index");
            }
        }
        public ActionResult GetPermanentEmployees(bool ispermanent)
        {
            var e = _repo.GetPermanentEmployees(ispermanent);
            if (e != null)
            {
                ViewBag.Message = e.Count();
                return View("Index");
            }
            else
            {
                ViewBag.Message = "Employee not found";
                return View("Index");
            }
        }
        public ActionResult GetEmployeesBetweenSalary(int s1, int s2)
        {
            var e = _repo.GetEmployeesBetweenSalary(s1, s2);
            if (e != null)
            {
                ViewBag.Message = e.Count();
                return View("Index");
            }
            else
            {
                ViewBag.Message = "Employee not found";
                return View("Index");
            }
        }
        public int AddEmployee(Employee emp)
        {
            _emp.Add(emp);
            return emp.ID;
        }
        public int UpdateEmployee(Employee emp)
        {
            var e = _emp.FirstOrDefault(x => x.ID == emp.ID);
            if (e != null)
            {
                e.Name = emp.Name;
                e.Salary = emp.Salary;
                e.Age = emp.Age;
                e.IsPermanent = emp.IsPermanent;
                e.DeptID = emp.DeptID;
                e.DateOfJoining = emp.DateOfJoining;
                return e.ID;
            }
            else
            {
                return -1;
            }
        }
        public int DeleteEmployee(int id)
        {
            var e = _emp.FirstOrDefault(x => x.ID == id);
            if (e != null)
            {
                _emp.Remove(e);
                return e.ID;
            }
            else
            {
                return -1;
            }
        }
    }
}
```

Code Explanation

- **Route Parameters**: Each action method accepts parameters (id, a1, a2, d1, d2, ispermanent, s1, s2) from the URL.
- **Data Retrieval**: The controller calls the repository methods to fetch data.
- **Conditional Logic**: Checks if data exists before displaying it.
- **Data Passing**: Uses ViewBag.Message to pass data to the view.
- **Consistent Pattern**: The same pattern is used for all actions.

Testing the Endpoints

Run the application and navigate to the URL path. Provide valid values. If the employee exists, the details will be displayed. If the employee does not exist, a message indicating that the employee is not found will be displayed.

```
http://localhost:7154/Employee/GetEmployeesBetweenSalary/5000/8000
```

Example to Understand Query Strings in ASP.NET Core Web API

Create a new Web **Application (Model View Controller)** project with the name **QueryStringDemo**.

Define the Employee Model

Create a folder by name **Models** and add a **Employee.cs** Model file and then copy and paste the following code.

```
namespace QueryStringDemo.Models
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public int Salary { get; set; }
        public int Age { get; set; }
        public bool IsPermanent { get; set; }
        public int DeptID { get; set; }
        public DateTime? DateOfJoining { get; set; }
        public static List<Employee> GetEmployeeDetails()
        {
            return new List<Employee>
            {
                new Employee { ID = 1, Name = "Rajesh", Salary = 5000, Age = 30, IsPermanent = true, DeptID = 1, DateOfJoining = DateTime.Parse("01/01/2010") },
                new Employee { ID = 2, Name = "Rakesh", Salary = 6000, Age = 32, IsPermanent = false, DeptID = 2, DateOfJoining = DateTime.Parse("02/02/2011") },
                new Employee { ID = 3, Name = "Suresh", Salary = 7000, Age = 34, IsPermanent = true, DeptID = 3, DateOfJoining = DateTime.Parse("03/03/2012") },
                new Employee { ID = 4, Name = "Ganesh", Salary = 8000, Age = 36, IsPermanent = false, DeptID = 4, DateOfJoining = DateTime.Parse("04/04/2013") },
                new Employee { ID = 5, Name = "Vijay", Salary = 9000, Age = 38, IsPermanent = true, DeptID = 5, DateOfJoining = DateTime.Parse("05/05/2014") }
            };
        }
    }
}
```

Initialize in Memory Data

Create a folder by name **RepositoryData** and add the **Models** folder and then copy and paste the following code.

```
namespace QueryStringDemo.RepositoryData
{
    public class EmployeeRepository
    {
        private static IEnumerable<Employee> _emp = Employee.GetEmployeeDetails();
        public IEnumerable<Employee> GetAllEmployees()
        {
            return _emp;
        }
        public Employee GetEmployeeByID(int id)
        {
            return _emp.FirstOrDefault(e => e.ID == id);
        }
        public IEnumerable<Employee> GetEmployeesByDeptID(int deptid)
        {
            return _emp.Where(e => e.DeptID == deptid);
        }
        public IEnumerable<Employee> GetEmployeesBetweenAge(int a1, int a2)
        {
            return _emp.Where(e => e.Age > a1 && e.Age < a2);
        }
        public IEnumerable<Employee> GetEmployeesBetweenDateOfJoining(DateTime? d1, DateTime? d2)
        {
            return _emp.Where(e => e.DateOfJoining > d1 && e.DateOfJoining < d2);
        }
        public IEnumerable<Employee> GetPermanentEmployees(bool ispermanent)
        {
            return _emp.Where(e => e.IsPermanent == ispermanent);
        }
        public IEnumerable<Employee> GetEmployeesBetweenSalary(int s1, int s2)
        {
            return _emp.Where(e => e.Salary > s1 && e.Salary < s2);
        }
        public int AddEmployee(Employee emp)
        {
            _emp.Add(emp);
            return emp.ID;
        }
        public int UpdateEmployee(Employee emp)
        {
            var e = _emp.FirstOrDefault(x => x.ID == emp.ID);
            if (e != null)
            {
                e.Name = emp.Name;
                e.Salary = emp.Salary;
                e.Age = emp.Age;
                e.IsPermanent = emp.IsPermanent;
                e.DeptID = emp.DeptID;
                e.DateOfJoining = emp.DateOfJoining;
                return e.ID;
            }
            else
            {
                return -1;
            }
        }
        public int DeleteEmployee(int id)
        {
            var e = _emp.FirstOrDefault(x => x.ID == id);
            if (e != null)
            {
                _emp.Remove(e);
                return e.ID;
            }
            else
            {
                return -1;
            }
        }
    }
}
```

Fetching Employees by ID using Query Strings

Add a new Controller by name **EmployeeQueryStringController** to the **Controllers** folder and then copy and paste the following code. In the action method, we are using the parameter **id** to get the employee details. In the action method, we are using the **EmployeeRepository** class to get the employee details. We are using the **ViewBag** to pass the employee data to the view.

```
namespace QueryStringDemo.Controllers
{
    public class EmployeeQueryStringController : Controller
    {
        EmployeeRepository _repo = new EmployeeRepository();
        public IEnumerable<Employee> _emp = Employee.GetEmployeeDetails();
        // GET: /EmployeeQueryString/
        public ActionResult Index(int id = 0)
        {
            if (id > 0)
            {
                var e = _repo.GetEmployeeByID(id);
                if (e != null)
                {
                    ViewBag.Message = e.ID + " " + e.Name + " " + e.Salary + " " + e.Age + " " + e.IsPermanent + " " + e.DeptID + " " + e.DateOfJoining;
                }
                else
                {
                    ViewBag.Message = "Employee not found";
                }
            }
            else
            {
                ViewBag.Message = "Please pass a valid Employee id";
            }
            return View();
        }
    }
}
```

Employee Controller

We always modify the Employee Controller code as shown below.

```
namespace QueryStringDemo.Controllers
{
    public class EmployeeController : Controller
    {
        EmployeeRepository _repo = new EmployeeRepository();
        public IEnumerable<Employee> _emp = Employee.GetEmployeeDetails();
        // GET: /Employee/
        public ActionResult Index(int id = 0)
        {
            if (id > 0)
            {
                var e = _repo.GetEmployeeByID(id);
                if (e != null)
                {
                    ViewBag.Message = e.ID + " " + e.Name + " " + e.Salary + " " + e.Age + " " + e.IsPermanent + " " + e.DeptID + " " + e.DateOfJoining;
                }
                else
                {
                    ViewBag.Message = "Employee not found";
                }
            }
            else
            {
                ViewBag.Message = "Please pass a valid Employee id";
            }
            return View();
        }
        public ActionResult GetEmployeeByID(int id)
        {
            var e = _repo.GetEmployeeByID(id);
            if (e != null)
            {
                ViewBag.Message = e.ID + " " + e.Name + " " + e.Salary + " " + e.Age + " " + e.IsPermanent + " " + e.DeptID + " " + e.DateOfJoining;
                return View("Index");
            }
            else
            {
                ViewBag.Message = "Employee not found";
                return View("Index");
            }
        }
        public ActionResult GetEmployeesByDeptID(int deptid)
        {
            var e = _repo.GetEmployeesByDeptID(deptid);
            if (e != null)
            {
                ViewBag.Message = e.Count();
                return View("Index");
            }
            else
            {
                ViewBag.Message = "Employee not found";
                return View("Index");
            }
        }
        public ActionResult GetEmployeesBetweenAge(int a1, int a2)
        {
            var e = _repo.GetEmployeesBetweenAge(a1, a2);
            if (e != null)
            {
                ViewBag.Message = e.Count();
                return View("Index");
            }
            else
            {
                ViewBag.Message = "Employee not found";
                return View("Index");
            }
        }
        public ActionResult GetEmployeesBetweenDateOfJoining(DateTime? d1, DateTime? d2)
        {
            var e = _repo.GetEmployeesBetweenDateOfJoining(d1, d2);
            if (e != null)
            {
                ViewBag.Message = e.Count();
                return View("Index");
            }
            else
            {
                ViewBag.Message = "Employee not found";
                return View("Index");
            }
        }
        public ActionResult GetPermanentEmployees(bool ispermanent)
        {
            var e = _repo.GetPermanentEmployees(ispermanent);
            if (e != null)
            {
                ViewBag.Message = e.Count();
                return View("Index");
            }
            else
            {
                ViewBag.Message = "Employee not found";
                return View("Index");
            }
        }
        public ActionResult GetEmployeesBetweenSalary(int s1, int s2)
        {
            var e = _repo.GetEmployeesBetweenSalary(s1, s2);
            if (e != null)
            {
                ViewBag.Message = e.Count();
                return View("Index");
            }
            else
            {
                ViewBag.Message = "Employee not found";
                return View("Index");
            }
        }
        public int AddEmployee(Employee emp)
        {
            _emp.Add(emp);
            return emp.ID;
        }
        public int UpdateEmployee(Employee emp)
        {
            var e = _emp.FirstOrDefault(x => x.ID == emp.ID);
            if (e != null)
            {
                e.Name = emp.Name;
                e.Salary = emp.Salary;
                e.Age = emp.Age;
                e.IsPermanent = emp.IsPermanent;
                e.DeptID = emp.DeptID;
                e.DateOfJoining = emp.DateOfJoining;
                return e.ID;
            }
            else
            {
                return -1;
            }
        }
        public int DeleteEmployee(int id)
        {
            var e = _emp.FirstOrDefault(x => x.ID == id);
            if (e != null)
            {
                _emp.Remove(e);
                return e.ID;
            }
            else
            {
                return -1;
            }
        }
    }
}
```

Code Explanation

- **Route Parameters**: Each action method accepts parameters (id, a1, a2, d1, d2, ispermanent, s1, s2) from the URL.
- **Data Retrieval**: The controller calls the repository methods to fetch data.
- **Conditional Logic**: Checks if data exists before displaying it.
- **Data Passing**: Uses ViewBag.Message to pass data to the view.
- **Consistent Pattern**: The same pattern is used for all actions.

Testing the Endpoints

Run the application and navigate to the URL path. Provide valid values. If the employee exists, the details will be displayed. If the employee does not exist, a message indicating that the employee is not found will be displayed.

```
http://localhost:7154/EmployeeQueryString/GetEmployeesBetweenSalary/5000/8000
```

Conclusion

Understanding route parameters and query strings is fundamental for building robust ASP.NET Core Web APIs. Route parameters provide clean, readable URLs for resource identification, while query strings offer flexible parameters for optional data. Both approaches have their place in modern web development, and choosing the right one depends on your specific requirements and user experience considerations.

Understanding and effectively utilizing Route Data and Query Strings are fundamental for building robust ASP.NET Core Web APIs. Route parameters provide clean, readable URLs for resource identification, while query strings offer flexible parameters for optional data. Both approaches have their place in modern web development, and choosing the right one depends on your specific requirements and user experience considerations.

Previous article
Routing in ASP.NET Core Web API

Next article
Multiple Office 365 + SharePoint Online Add-Ins in ASP.NET MVC

2 thoughts on "Route Parameters and Query Strings in Routing"

Sandeep
August 2, 2025 at 12:07 pm

Sir please upload Web API Routing Attribute and Custom Attribute

Reply

admin
August 2, 2025 at 12:07 pm

Sorry

Reply

Leave a Reply

Would you like to join the discussion?
Feel free to contribute!

Name *
Email *
Comment *

Post Comment